Application Estate Monitoring, Reporting & Supportability (Microsoft Graph & Power BI)

Implementation Solution Guide

Version 1.0.0 · 2026-07-20

Contents

Executive Summary 5

The situation . . . 5

The solution . . . 5

Why run it yourself . . . 5

Government tenants . . . 6

The bottom line . . . 6

Solution Overview 6

The problem this solves . . . 6

What this solution is NOT . . . 6

The measurement model . . . 7

The KPI set . . . 7

The SLO for application delivery . . . 8

Where the data lives . . . 8

The pipeline . . . 9

The supportability model . . . 9

Deployment model . . . 9

Government cloud . . . 10

What is in this solution . . . 10

Requirements 10

R1. Intune role that can read apps and device install status . . . 10

R2. Microsoft Entra app registration for unattended export (app-only auth) . . . 11

R3. Delegated Graph scopes for the read-only (interactive) scripts . . . 11

R4. Application Graph scopes for the unattended export identity . . . 11

R5. Power BI Pro (or Premium/Fabric) licensing . . . 12

R6. A storage location for the exports . . . 12

R7. Power BI Desktop for authoring the dataset and report . . . 13

R8. A Power BI workspace for the reporting artifacts . . . 13

R9. (Optional but recommended) Log Analytics workspace + Intune diagnostic settings . . . 13

R10. PowerShell 7 with the Microsoft Graph SDK . . . 14

R11. Government cloud endpoints (GCC High / DoD) — if applicable . . . 14

Global Requirements — Intune Solutions (category-level) 15

G1. Microsoft Entra ID tenant . . . 15

G2. Microsoft Intune subscription . . . 15

G3. Administrative roles . . . 15

G4. Device enrollment in Intune . . . 15

G5. Pilot security group . . . 15
G6. Supported Windows baseline . . . 15
G7. Network access to Microsoft cloud endpoints . . . 15
Implementation 16
Step 0. Select your cloud (Commercial vs Government) . . . 16
Step 1. Confirm the KPI set and SLO (do this on paper first) . . . 17
Step 2. Create the pilot reporting-scope group . . . 17
Step 3. Create the storage location . . . 17
Step 4. Create the export identity and grant least-privilege scopes . . . 18
Step 5. Run the first export by hand (pilot) . . . 19
Step 6. Validate the export against the console (the trust gate) . . . 19
Step 7. Author the Power BI dataset . . . 19
Step 8. Build the three report views . . . 20
Step 9. Publish, share, and schedule refresh . . . 21
Step 10. Turn on the nightly export . . . 21
Step 11. Configure alerting (Azure Monitor) . . . 22
Step 12. Stand up the supportability model . . . 22
Step 13. Expand to the whole estate . . . 23
Testing & Validation 23
Gate 1 — The extractor authenticates and returns data . . . 23
Gate 2 — Export counts match the Intune console (the trust gate) . . . 23
Gate 3 — The KPI snapshot math is correct . . . 24
Gate 4 — Top-failures ranking surfaces the right apps . . . 24
Gate 5 — Version drift reflects reality . . . 24
Gate 6 — The Power BI dataset refreshes from storage . . . 24
Gate 7 — The three views answer their questions . . . 25
Gate 8 — The nightly export runs unattended . . . 25
Gate 9 — An alert fires and reaches a human . . . 25
Gate 10 — The supportability loop closes . . . 25
Overall exit criteria (pilot → expand) . . . 26
Troubleshooting 26
T1. The extractor returns no rows / empty CSV . . . 26
T2. Export counts don't match the Intune console (Testing Gate 2 fails) . . . 26
T3. Power BI refresh fails with a credentials or gateway error . . . 27
T4. The nightly task prompts for sign-in or produces nothing (Testing Gate 8 fails) . . . 27
T5. Version-drift numbers look wrong or the app isn't found . . . 27

T6. Graph requests throttle at fleet scale (HTTP 429) . . . 28
T7. Alert never fires, or fires constantly (Testing Gate 9) . . . 28
T8. Government-cloud endpoint errors . . . 28
T9. Adoption numbers look implausible or can't be sourced . . . 29
T10. Two visuals show different values for the "same" KPI . . . 29
Best Practices 29
Alert vs. weekly review — the split that keeps the pager credible . . . 30
Ownership — every KPI has a name next to it . . . 30
The evidence pack — make every app-failure ticket actionable on first read . . . 31
Keep the dashboard honest . . . 31
Least privilege for the reporting identity . . . 31
Data retention . . . 32
Cost . . . 32
Government cloud . . . 32
Deliverables Summary 32
Documentation (Word + PDF) . . . 33
PowerShell scripts (ZIP) . . . 33
What you achieve . . . 34
Appendix A — PowerShell Scripts 35

Executive Summary

The situation

Your organization deploys dozens or hundreds of applications through Microsoft Intune. You can see whether one app installed on one device — but you cannot answer the questions leadership and auditors actually ask: *Which apps fail most? Is the fleet on the approved versions? How fast do rollouts reach everyone? Are we using what we license? When something breaks, who owns it?* Today those answers come from a manual CSV export pasted into a spreadsheet — different every time, out of date the moment it's made, and silent when the failure rate doubles overnight.

The solution

This package builds the **monitoring, reporting, and supportability layer** for your application estate. It delivers:

- **A fixed KPI set with healthy ranges** — install success and failure rate (by app and by deployment ring), detection mismatch, time-to-deploy, version drift and patch currency, adoption, and stale devices — each with one authoritative source so the same question always returns the same number.
- **A published service-level objective (SLO)** for application delivery — availability, timeliness, and currency targets, with an error budget that makes "good enough" measurable.
- **An automated pipeline** — a scheduled, least-privilege Microsoft Graph export into storage, refreshed into a Power BI dataset, so nobody exports by hand again.
- **A three-view dashboard** — an executive view (are we meeting our promises?), an operations view (where do I spend my day?), and a per-app drill-down (what's wrong with this app?), with one-click drill-through from a failing app to its affected devices.
- **Alerting that stays credible** — a page only for a live failure spike or an error-budget burn; everything else routed to a weekly or monthly review.
- **A supportability model** — every KPI has a named owner, a review cadence, and an evidence pack, so a red number becomes an actionable ticket instead of a debate.

Why run it yourself

You do not need a consultant. The package is self-guided, click-by-click documentation a competent administrator follows end to end, backed by five tested PowerShell scripts.

- **Simpler** — exact, ordered steps with full navigation paths; every object you create is named and reused through testing and troubleshooting.
- **Cheaper** — one fixed price instead of an open consulting meter that runs into thousands per day, and the pipeline itself is read-only and low-volume by design, so it stays cheap to operate.

- **Proven** — a pilot-first method (**Assess** → **Configure** → **Pilot** → **Validate** → **Expand**) with a hard trust gate: the export must match the Intune console before any number reaches a leader's screen.

Government tenants

Every step and script targets either the Commercial or the US Government cloud (GCC High / DoD) via a single parameter, with the endpoint substitution table included — so public-sector organizations build the same pipeline against their sovereign-cloud services.

The bottom line

A living, trustworthy picture of how well your applications are actually delivered, kept current, and used — refreshed automatically, alerting on what matters, and owned by name. Built once for a pilot, then widened to the whole estate, and reusable on every rollout that follows.

Solution Overview

The problem this solves

You can see whether **one** app installed on **one** device. What you cannot see is whether the **application estate as a whole** is healthy: which apps fail most often, whether failures cluster in one deployment ring, how long an app takes to reach 95% of its target population, how far behind the approved version the fleet has drifted, whether anyone is actually using the app you licensed, and — when something breaks — who owns the number and what evidence support must attach to the ticket.

Most organizations answer those questions by exporting a CSV from the Intune console every few weeks, pasting it into a spreadsheet, and arguing about which number is right. The export is manual, the definitions change every time, and nothing raises its hand when the failure rate doubles overnight.

This solution builds the **monitoring, reporting, and supportability layer** for application management. It fixes a KPI set with explicit healthy ranges, defines a delivery SLO, builds a **scheduled Microsoft Graph export** → **storage** → **Power BI** pipeline, ships a **three-view dashboard** (executive, operations, per-app drill-down), defines **what alerts versus what waits for the weekly review**, and puts an **operational review and triage model** around it so a number that goes red lands on a named owner's queue with the evidence already attached.

What this solution is NOT

- It is **not** app packaging or deployment authoring. It assumes apps are already being deployed.
- It is **not** per-incident app-install troubleshooting. When a specific install fails, this layer tells you *which* app, ring, and device to look at, and hands the evidence to the troubleshooter.
- It is **not** software licensing or spend reporting. Entitlement counts, cost per seat, renewal timelines, and reharvesting are a different discipline with a different data model. This layer measures **deployment health and supportability**, not money.

The measurement model

Every metric in this solution answers one of four questions. Keeping them separate is what stops a dashboard from becoming a wall of numbers nobody acts on.

Question	Metric family	Example
Did the app arrive?	**Delivery**	Install success rate, failure rate, pending rate
Did it arrive fast enough?	**Velocity**	Time-to-deploy (P50/P90), ring completion time
Is it the right version?	**Currency**	Version drift %, patch currency, supersedence lag
Is anyone using it?	**Value**	App adoption / active usage, uninstall success

A fifth family, **Data quality**, sits underneath all of them: detection mismatch rate and stale check-in rate tell you whether the other four numbers can be trusted at all.

The KPI set

The solution defines one authoritative KPI set, named `App Estate KPI Set – v1`, and every dashboard visual, alert rule, and script output traces back to it. Each KPI has one — and only one — source of truth, so the same question always returns the same number.

KPI	Definition	Healthy range	Source
Install success rate	`installed ÷ (installed + failed)` for Required assignments	≥ 97%	Graph `deviceStatuses`
Failure rate by app	`failed ÷ applicable` per app	≤ 3%	Graph `deviceStatuses`
Failure rate by ring	Same, grouped by deployment ring group	≤ 3%, and no ring > 2× another	Graph + group membership
Detection mismatch rate	Devices reporting install success but not detected	≤ 1%	Graph install state vs. detected app inventory
Time-to-deploy (P50 / P90)	Hours from assignment creation to 50% / 90% of target installed	P50 ≤ 24 h, P90 ≤ 96 h	Export history (trend)
Version drift %	Devices not on the approved version ÷ devices with the app	≤ 10%	Graph `detectedApps`
Patch currency (n-1)	Devices on the current or previous approved version	≥ 95%	Graph `detectedApps`

KPI	Definition	Healthy range	Source
Uninstall success rate	`installed`→`removed` success for Uninstall assignments	≥ 95%	Graph `deviceStatuses`
App adoption / usage	Devices with a launch/active signal in 30 days ÷ devices with the app installed	≥ 70% (review below)	Usage telemetry (see below)
Stale device rate	Devices whose last check-in is > 7 days	≤ 5%	Graph `managedDevices`

> ***Honest caveat on adoption:*** *Intune does **not** report per-app *launch* telemetry for Windows desktop applications. Adoption is therefore assembled from whichever signal your estate actually has — Microsoft 365 Apps usage reports, Defender for Endpoint software inventory, Windows Update-for-Business / Endpoint analytics app reliability data, or the vendor's own portal. The solution tells you how to wire whichever source you have and how to label the KPI's confidence. Do not present an adoption number whose source you cannot name.*

The SLO for application delivery

A KPI tells you where you are. An **SLO** tells you what you promised. The solution defines the app delivery SLO as three commitments, published to the business and tracked on the executive view:

1. **Availability** — 97% of Required assignments reach *Installed and Detected* on in-scope devices.
2. **Timeliness** — 90% of in-scope devices install a new Required app within **96 hours** of assignment (measured after the ring's scheduled availability time, not the authoring time).
1. **Currency** — 95% of devices with a managed app are on the current or previous approved version.

The **error budget** is the inverse: at a 97% availability objective, 3% of applicable installs may fail in a rolling 30-day window. Burning more than half the budget in the first third of the window is itself an alert condition (see Alerting) — it is the earliest honest signal that a rollout is going wrong.

Where the data lives

Five sources, in order of how much you should rely on them:

1. **Microsoft Graph —** `deviceAppManagement` (`mobileApps`, `.../deviceStatuses`, `.../userStatuses`, `mobileAppRelationships`). The authoritative per-device install and detection state. This is the backbone of the pipeline.
1. **Microsoft Graph —** `deviceManagement` (`managedDevices`, `detectedApps`, `managedDevices/{id}/detectedApps`, `reports/exportJobs`). Fleet inventory, installed application versions (the basis for version drift), and the asynchronous **export jobs** API that produces the same payloads the console's report pages show.
1. **Intune admin center reports** — the on-screen views you validate the pipeline against:

Intune admin center > Reports > Device management > App install status, and per app at **Intune admin center > Apps > All apps > (app) > Monitor > Device install status**.

1. **Log Analytics / Azure Monitor** — Intune **diagnostic settings** stream operational logs

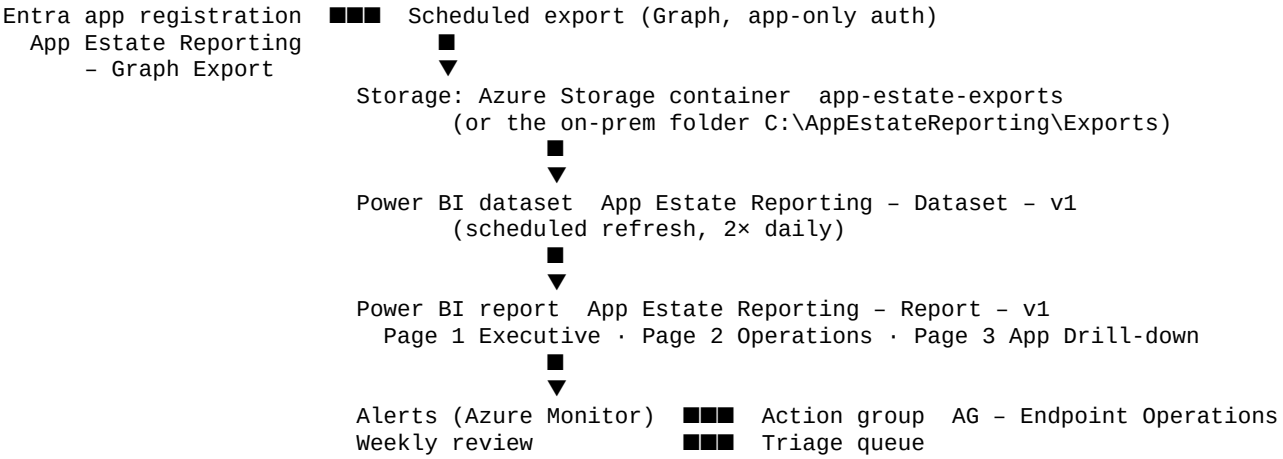
(`DeviceComplianceOrg`, `Devices`, `AuditLogs`, and where available `Apps` / `AppWorkload` categories) into a workspace where they can be queried with KQL, retained, and alerted on.

1. **Intune Data Warehouse** (OData feed, `reports/dataWarehouseFeeds`) — a historical, daily-refreshed

model useful for long-run trend analysis. It is the slowest-moving source; treat it as a supplement for trends, not the live operational picture.

> ***Choose one per KPI.*** *The most common cause of a distrusted dashboard is the same KPI computed from two sources that disagree by a few percent because they count "applicable" differently. The KPI table above assigns exactly one source to each metric. Keep it that way.*

The pipeline



Every object in that diagram is created by name in the Implementation section and the exact same name is reused in Testing, Troubleshooting, and the scripts.

The supportability model

Reporting without ownership produces dashboards nobody reads. This solution pairs each KPI with:

- **A named owner** — the role accountable for the number moving in the right direction.
- **A cadence** — daily triage, weekly operational review, or monthly service review.
- **A trigger** — the threshold at which it becomes work.
- **An evidence pack** — the exact artifacts support attaches to an app-failure ticket, so the ticket

is actionable on first read instead of bouncing back for more information.

Deployment model

The solution follows the standard model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.** You build the pipeline against a small set of apps first, prove the numbers match the console, and only then widen it

to the whole estate and turn on alerting.

Government cloud

US Government tenants (GCC, GCC High, DoD) use **different service endpoints** for Graph, the admin portals, and Power BI. Every step and every script in this solution is written to target either cloud; the endpoint substitution table and the parity caveats are in Requirements (R11) and Implementation (Step 0). Nothing in the method changes — only the URLs and one script parameter.

What is in this solution

- **This overview** — the measurement model, KPI set, SLO, data sources, pipeline, supportability model.
- **Requirements** (`02-requirements.md`) — licensing, roles, Graph scopes, Power BI, Azure, network.
- **Implementation** (`03-implementation.md`) — the exact ordered build of the pipeline and dashboard.
- **Testing** (`05-testing.md`) — the pass/fail gates that prove the numbers are trustworthy.
- **Troubleshooting** (`06-troubleshooting.md`) — what to do when the pipeline or dashboard misbehaves.
- **Best practices** (`07-best-practices.md`) — retention, least privilege, cost, and review discipline.
- **Deliverables summary** (`08-deliverables-summary.md`) and the **executive summary**.
- **Five PowerShell scripts** — install-status extractor, top-N failure report, version-drift report, KPI snapshot generator, and the scheduled-export bootstrap.

Requirements

Before you build the pipeline or run the scripts, confirm the items below. The **category-level global requirements** (Entra tenant, Intune subscription, enrolled devices, supported Windows baseline, network access) are in `global-requirements-intune-solutions.md` and are assumed here. This file covers the requirements specific to **monitoring, reporting, and supportability for the application estate**.

Each requirement is written as **Description / Why / Verify / Implement if missing.** Where a Microsoft Graph permission is needed, the **exact scope string** is given. Where a paid SKU is needed, it is called out so there are no surprises.

R1. Intune role that can read apps and device install status

Description: An Entra/Intune role granting **read** access to **Apps**, their **assignments**, and per-device **install status** — plus device inventory. **Intune Administrator** covers everything; **Help Desk Operator**, or a custom role with **Managed apps** → **Read** and **Organization** → **Read**, is enough for the read-only extractor, report, drift, and KPI-snapshot scripts.

Why: Every metric reads app configuration, install/detection state, and detected-app inventory. Without app-and-device read rights the console shows nothing and the scripts return empty results.

Verify: Sign in to `https://intune.microsoft.com`, open **Apps** → **All apps**, open any app → **Monitor** → **Device install status**. If you see the app list and per-device status, you have read access. For roles: **Tenant administration** → **Roles** → **All roles** and confirm your assignment.

Implement: Tenant administration → **Roles** → **All roles** → select **Help Desk Operator** (read) or **Intune Administrator** → **Assignments** → **Assign** → add your account and a scope group.

R2. Microsoft Entra app registration for unattended export (app-only auth)

Description: A dedicated Entra **app registration**, named `App Estate Reporting – Graph Export`, with a client secret or (preferred) a certificate, used by the scheduled export to authenticate to Graph **without a signed-in user**.

Why: A scheduled task cannot rely on an interactive `Connect-MgGraph` sign-in. App-only (client-credentials) authentication with **application permissions** is what lets the export run unattended overnight. Naming it consistently is what lets you audit and revoke it later.

Verify: Entra → **App registrations** → **All applications**; the registration `App Estate Reporting – Graph Export` exists and shows a valid credential under **Certificates & secrets** that has not expired.

Implement: Entra → **App registrations** → **New registration** → name it exactly `App Estate Reporting – Graph Export` → **Register**. Then **Certificates & secrets** → **New client secret** (or upload a certificate) and record the value in your secret store. The bootstrap script (`APPRPT-Bootstrap-ScheduledExport.ps1`) creates this registration and grants the scopes in R4 for you; do it by hand only if you cannot run the script as a Global Administrator.

R3. Delegated Graph scopes for the read-only (interactive) scripts

Description: For the four read-only scripts run interactively by an operator, delegated Microsoft Graph permissions: `DeviceManagementApps.Read.All` and `DeviceManagementManagedDevices.Read.All`.

Why: The extractor, report, drift, and KPI-snapshot scripts read app definitions, assignments, per-device install/detection state, and detected-app inventory through Graph. These are read scopes; none of the four writes anything.

Verify: Run any read-only script; at the first `Connect-MgGraph` prompt, sign in and **consent** to the listed scopes. If it returns data, access works. Confirm granted delegated permissions in Entra → **Enterprise applications** → **Microsoft Graph Command Line Tools** → **Permissions**.

Implement: `Install-Module Microsoft.Graph -Scope CurrentUser`. Each script calls `Connect-MgGraph` with the scopes it needs; an Intune Administrator can consent on first use. If your tenant requires admin consent for the Graph Command Line Tools app, have a Global Administrator grant it once.

R4. Application Graph scopes for the unattended export identity

Description: For the app registration in R2, **application** (not delegated) Microsoft Graph permissions, admin-consented: `DeviceManagementApps.Read.All` and `DeviceManagementManagedDevices.Read.All`. If you also drive the Graph **exportJobs** report API from the service identity, that same `DeviceManagementApps.Read.All` / `DeviceManagementManagedDevices.Read.All` pair covers it.

Why: Application permissions are what let the export identity read the whole estate unattended. Granting only **read** scopes to the automation identity is the least-privilege posture for a reporting pipeline — it never needs to write.

Verify: Entra → **App registrations** → `App Estate Reporting – Graph Export` → **API permissions**; both scopes are listed as **Application** type with a green **Granted for <tenant>** status.

Implement: App registration → **API permissions** → **Add a permission** → **Microsoft Graph** → **Application permissions** → add the two scopes → **Grant admin consent for <tenant>**. The bootstrap script does this for you.

R5. Power BI Pro (or Premium/Fabric) licensing

Description: Each person who **publishes or refreshes** the dataset and **consumes** the report in a Power BI workspace needs a **Power BI Pro** license (included in Microsoft 365 E5; otherwise a per-user add-on). Sharing to a large read-only audience without per-user Pro requires **Power BI Premium** capacity or a **Microsoft Fabric** capacity (F-SKU).

Why: Scheduled dataset refresh, workspace sharing, and app distribution are Pro/Premium features. Power BI Desktop is free for authoring, but publishing to the service and scheduling refresh is not.

Verify: In `https://app.powerbi.com` your account can **create a workspace** and, on a dataset, open **Settings** → **Scheduled refresh**. If those are available, licensing is sufficient for a small team.

Implement: Assign **Power BI Pro** to the pipeline owner and each report consumer (Microsoft 365 admin center → **Users** → **Licenses**), or provision **Premium/Fabric** capacity for broad distribution. **Government note:** Power BI has a **separate US Government cloud** — see R11.

R6. A storage location for the exports

Description: A destination the scheduled export writes CSV/JSON to and Power BI reads from. Default: an **Azure Storage** account with a blob container named `app-estate-exports`. Alternative for a self-hosted runner: a local/UNC folder `C:\AppEstateReporting\Exports`.

Why: The pipeline decouples *extraction* (runs on a schedule) from *presentation* (Power BI refresh). A durable, access-controlled storage layer between them is what makes the numbers reproducible and auditable, and lets Power BI refresh without re-querying Graph.

Verify: For Azure Storage: `https://portal.azure.com` → the storage account → **Containers** → `app-estate-exports` exists. For a folder: the path exists and the task's service account can write to it.

Implement: Azure portal → **Storage accounts** → **Create** (or reuse one) → **Containers** → **+ Container** → name `app-estate-exports`, private access. Grant the export identity **Storage Blob Data Contributor** on that container, and the Power BI refresh identity **Storage Blob Data Reader**. For the folder option, create `C:\AppEstateReporting\Exports` and restrict its ACL to the task account and the reporting

owners.

R7. Power BI Desktop for authoring the dataset and report

Description: The free **Power BI Desktop** app installed on the authoring workstation.

Why: The dataset (`App Estate Reporting – Dataset – v1`) and report (`App Estate Reporting – Report – v1`) are authored in Desktop, then published to the service.

Verify: Power BI Desktop opens and can connect to your storage location via **Get data**.

Implement: Install from the Microsoft Store or `https://powerbi.microsoft.com/desktop`. Keep it current; the connectors for Azure Blob and folder sources ship in-box.

R8. A Power BI workspace for the reporting artifacts

Description: A dedicated Power BI **workspace** named `Endpoint & App Reporting` that holds the dataset, report, and (optionally) a published app.

Why: A named workspace is the unit of access control and scheduled refresh. Publishing into "My workspace" cannot be shared or governed properly.

Verify: `https://app.powerbi.com` → **Workspaces** → `Endpoint & App Reporting` exists and you are at least a **Member**.

Implement: Power BI service → **Workspaces** → **New workspace** → name `Endpoint & App Reporting` → add the reporting owners as **Members**, consumers via a published app or **Viewer** role.

R9. (Optional but recommended) Log Analytics workspace + Intune diagnostic settings

Description: An Azure **Log Analytics** workspace named `law-endpoint-ops` with Intune **diagnostic settings** streaming operational logs to it, used for retention, KQL queries, and Azure Monitor alerts.

Why: The Graph install-status data is a point-in-time snapshot. Diagnostic-setting logs give you **history and alerting** — the ability to say "failure rate doubled at 02:00" and to page someone automatically. This is what turns reporting into monitoring.

Verify: `https://portal.azure.com` → Log Analytics workspace `law-endpoint-ops` exists; and Intune admin center → **Tenant administration** → **Diagnostic settings** shows a setting named `Intune → law-endpoint-ops` sending logs.

Implement: Azure portal → **Log Analytics workspaces** → **Create** → name `law-endpoint-ops`. Then Intune admin center → **Tenant administration** → **Diagnostic settings** → **+ Add diagnostic setting** → name it `Intune → law-endpoint-ops` → select the log categories available in your tenant (at minimum `AuditLogs`, `DeviceComplianceOrg`, `Devices`; add app/operational categories where your tenant exposes them) → **Destination: Send to Log Analytics workspace** → **law-endpoint-ops** → **Save**. Log ingestion and retention are billed by Azure Monitor (see Best Practices → cost).

R10. PowerShell 7 with the Microsoft Graph SDK

Description: A workstation (and the export runner) running **PowerShell 7** with the **Microsoft.Graph** module installed. PowerShell 5.1 works for the interactive scripts but PowerShell 7 is recommended for the scheduled runner.

Why: All five scripts use the Microsoft Graph PowerShell SDK. The bootstrap and export scripts use app-only authentication that is most reliable on PowerShell 7.

Verify: `$PSVersionTable.PSVersion` (expect 7.x) and `Get-Module Microsoft.Graph -ListAvailable` (expect a version listed).

Implement: Install PowerShell 7 from `https://aka.ms/powershell`, then `Install-Module Microsoft.Graph -Scope CurrentUser`. Run scripts with `pwsh -File .\<script-name>.ps1`.

R11. Government cloud endpoints (GCC High / DoD) — if applicable

Description: If your tenant is in a **US Government** cloud, every service URL differs, and one script parameter (`-Environment`) selects the cloud. The substitution table:

Service	Commercial	GCC High	DoD
Microsoft Graph	`graph.microsoft.com`	`graph.microsoft.us`	`dod-graph.microsoft.us`
`Connect-MgGraph -Environment`	`Global`	`USGov`	`USGovDoD`
Intune admin center	`intune.microsoft.com` / `endpoint.microsoft.com`	`endpoint.microsoft.us`	`endpoint.microsoft.us`
Azure portal	`portal.azure.com`	`portal.azure.us`	`portal.azure.us`
Entra sign-in authority	`login.microsoftonline.com`	`login.microsoftonline.us`	`login.microsoftonline.us`
Power BI service	`app.powerbi.com`	`app.powerbigov.us`	`app.powerbigov.us`

Why: Sovereign clouds are network-isolated. A script pointed at `graph.microsoft.com` from a GCC High tenant fails to authenticate; a Power BI report built against the commercial service cannot be published to the Government cloud. Endpoint parity also lags — some Graph report endpoints and Power BI features arrive in the Government cloud later than in Commercial.

Verify: Confirm your cloud in the Azure portal (**Microsoft Entra ID** → **Overview** shows the cloud instance). Reach the Government Graph endpoint from the runner, e.g. `Connect-MgGraph -Environment USGov` returns a context.

Implement: Pass `-Environment USGov` (or `USGovDoD`) to every script; the scripts route `Connect-MgGraph` and the Graph base URL accordingly. Author the Power BI dataset against the Government storage endpoint and publish to `app.powerbigov.us`. Where a Graph report endpoint or a Power BI feature is not yet available in your Government cloud, **verify for GCC High** against Microsoft's US-sovereign-cloud documentation before relying on it — do not assume commercial parity.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

Implementation

Build the pipeline in this order. Do **not** skip ahead to the whole fleet — the pilot (Steps 5–6) proves the numbers before you widen scope. Every object you create is given an **exact name**; reuse that name verbatim in later steps, in Testing, in Troubleshooting, and in the scripts.

Naming convention used throughout (`<Purpose> – <Scope> – <Stage>`):

Object	Exact name
Entra app registration (export identity)	`App Estate Reporting – Graph Export`
Entra security group (pilot reporting scope)	`PILOT – App Estate Reporting`
Azure Storage container	`app-estate-exports`
Local export folder (self-hosted option)	`C:\AppEstateReporting\Exports`
Log Analytics workspace	`law-endpoint-ops`
Intune diagnostic setting	`Intune → law-endpoint-ops`
Power BI workspace	`Endpoint & App Reporting`
Power BI dataset	`App Estate Reporting – Dataset – v1`
Power BI report	`App Estate Reporting – Report – v1`
KPI set (documented)	`App Estate KPI Set – v1`
Scheduled task (export runner)	`App Estate Reporting – Nightly Export`
Azure Monitor action group	`AG – Endpoint Operations`
Azure Monitor alert rule (example)	`ALERT – App Failure Rate – High`

Step 0. Select your cloud (Commercial vs Government)

Before anything else, decide which cloud your tenant is in, because it changes every URL and one script parameter.

- **Commercial:** portals at `intune.microsoft.com` / `portal.azure.com`, Graph at `graph.microsoft.com`, Power BI at `app.powerbi.com`. Scripts run with `-Environment Global` (the default).
- **GCC High:** portals at `endpoint.microsoft.us` / `portal.azure.us`, Graph at `graph.microsoft.us`,

Power BI at `app.powerbigov.us`. Scripts run with `-Environment USGov`.

• **DoD:** as GCC High but Graph at `dod-graph.microsoft.us`; scripts run with `-Environment USGovDoD`.

The full substitution table is in Requirements R11. In every step below, use **your** cloud's URL. All five scripts accept `-Environment` and route both `Connect-MgGraph` and the Graph base URL for you.

Step 1. Confirm the KPI set and SLO (do this on paper first)

1. Open `01-solution-overview.md` and read the **KPI set** table and the **SLO** section.
2. Confirm the three SLO commitments with whoever owns the endpoint service (availability 97%,

timeliness 90% in 96 h, currency 95% on n-1). Adjust the numbers to your reality **now**, before you build visuals against them — changing a target later means re-authoring alert thresholds.

1. Record the agreed numbers as the KPI set `App Estate KPI Set – v1` in your team wiki. Every

later object references this by name.

Why here: the dashboard is only as useful as the definitions behind it. Fixing the definitions first is what stops two people reading the same chart and computing "success" differently.

Step 2. Create the pilot reporting-scope group

1. Go to **Entra admin center (**`entra.microsoft.com`**) > Identity > Groups > All groups > New group**.
2. **Group type:** Security. **Group name:** `PILOT – App Estate Reporting`.

Membership type: Assigned.

1. **No members yet** → **Create.**
2. Open the group and add **5–10 representative devices** (a mix of your deployment rings) under

Members → **Add members**. This is the population the pilot exports and dashboard cover.

Why: you validate the whole pipeline against a small, known set of devices whose real install state you can read in the console by hand. Fixing definitions against a control group is far cheaper than against the whole fleet.

Step 3. Create the storage location

Azure Storage (default):

1. **Azure portal (**`portal.azure.com`**) > Storage accounts > Create** (or open an existing account).
2. Once the account exists, open it → **Data storage > Containers > + Container**.
3. **Name:** `app-estate-exports`. **Public access level:** Private (no anonymous access) → **Create**.
4. Grant access with least privilege under the container → **Access Control (IAM) > Add role

assignment**:

• **Storage Blob Data Contributor** → the export identity `App Estate Reporting – Graph Export` (added after Step 4) *and* the interactive operator who runs the extractor.

• **Storage Blob Data Reader** → the Power BI refresh identity / gateway account.

Self-hosted folder (alternative): create `C:\AppEstateReporting\Exports` on the runner and restrict its NTFS ACL to the scheduled-task service account (read/write) and the reporting owners (read).

Why: decoupling extraction from presentation through durable storage is what makes the numbers reproducible — Power BI refreshes from a stored snapshot instead of hammering Graph, and you keep an auditable history of exactly what each report was built from.

Step 4. Create the export identity and grant least-privilege scopes

The fastest and least error-prone way is the bootstrap script; the manual path is given as a fallback.

Scripted (recommended):

1. On the authoring workstation, open **PowerShell 7** as an account that is **Global Administrator** (needed once to grant admin consent).

1. Run:

```
pwsh -File .\APPRPT-Bootstrap-ScheduledExport.ps1 -Environment Global -WhatIf
```

Review the plan (it prints the registration name, the two application scopes from R4, and the scheduled task it will create). Then run it **without** `-WhatIf` to apply.

1. The script creates the app registration `App Estate Reporting – Graph Export`, adds a certificate credential, grants and admin-consents `DeviceManagementApps.Read.All` and `DeviceManagementManagedDevices.Read.All` (Application), and registers the scheduled task `App Estate Reporting – Nightly Export`.

1. Record the printed **Application (client) ID**, **Tenant ID**, and certificate thumbprint in your secret store.

Manual (fallback):

1. **Entra admin center > Identity > Applications > App registrations > New registration.**

2. **Name:** `App Estate Reporting – Graph Export`. **Supported account types:** single tenant → **Register**.

1. **Certificates & secrets > Upload certificate** (preferred) or **New client secret**; record it.

2. **API permissions > Add a permission > Microsoft Graph > Application permissions** → add `DeviceManagementApps.Read.All` and `DeviceManagementManagedDevices.Read.All` → **Grant admin consent for <tenant>** (both show green).

1. Return to Step 3 and assign this registration **Storage Blob Data Contributor** on `app-estate-exports`.

Why: app-only auth with **read-only** application scopes is the correct least-privilege posture for an unattended reporting pipeline — it can read the whole estate but can never change a policy, assignment, or device.

Step 5. Run the first export by hand (pilot)

1. Open **PowerShell 7** as the interactive operator (delegated scopes from R3).
2. Extract install status for the pilot apps:

```
pwsh -File .\APPRPT-Export-AppInstallStatus.ps1 -Environment Global `
     -OutputPath C:\AppEstateReporting\Exports -Format Both
```

This connects to Graph, reads every app's `deviceStatuses`, and writes `app-install-status_<yyyyMMdd-HHmm>.csv` and `.json` to the output path (or, with `-BlobContainer app-estate-exports`, straight to Azure Storage).

1. Generate the supporting datasets:

```
pwsh -File .\APPRPT-Report-TopFailures.ps1     -Environment Global -Top 20 -OutputPath <path>
pwsh -File .\APPRPT-Report-VersionDrift.ps1    -Environment Global -OutputPath <path>
pwsh -File .\APPRPT-Generate-KPISnapshot.ps1   -Environment Global -OutputPath <path>
```

1. Open the CSVs. Confirm the row counts look sane and the pilot apps you know about appear.

Why: a manual run proves the identity, scopes, and storage all work before you automate. If the scheduled task later produces nothing, you already know the moving parts are sound.

Step 6. Validate the export against the console (the trust gate)

This is the most important step. **Do not build the dashboard until the export matches the console.**

1. Pick one pilot app. In **Intune admin center > Apps > All apps > (the app) > Monitor > Device

install status**, note the counts: Installed, Failed, Not applicable, Pending.

1. Open `app-install-status_*.csv`, filter to that app, and count the same states.
2. They should match (allow for a small drift if devices checked in between the two reads). If they

diverge materially, resolve it now — the usual cause is counting "applicable" differently (see Troubleshooting) — before any number reaches a leader's screen.

1. Cross-check one KPI: open the KPI snapshot CSV, read **Install success rate** for that app, and

confirm it equals `installed ÷ (installed + failed)` from the console counts.

Pass gate: console counts and export counts agree for the pilot app, and the KPI snapshot's success rate matches the hand calculation. Only then proceed.

Step 7. Author the Power BI dataset

1. Open **Power BI Desktop** → **Get data**.

• **Azure Storage:** choose **Azure > Azure Blob Storage**, enter the account, select the

`app-estate-exports` container.

• **Folder:** choose **Folder**, point at `C:\AppEstateReporting\Exports`.

1. In **Power Query**, combine the `app-install-status_*` files into one table; set data types (dates as Date/Time, counts as Whole Number). Do the same for the drift and KPI-snapshot files as separate queries.

1. Add a small **KPI targets** table (enter data manually) holding the numbers from Step 1: `MetricName`, `Target`, `Direction` (higher-is-better / lower-is-better). This is what the visuals compare against, so a target change is a one-cell edit.

1. Build the model: relate install-status and drift by **AppId**; relate device rows to a **Ring** column derived from group membership (add a query that maps device → ring group).

1. Create the core **measures** (DAX), each mapping to `App Estate KPI Set – v1`:

• `Install Success Rate = DIVIDE([Installed], [Installed] + [Failed])`

• `Failure Rate = DIVIDE([Failed], [Applicable])`

• `Detection Mismatch Rate = DIVIDE([InstalledButNotDetected], [Installed])`

• `Version Drift % = DIVIDE([DevicesOffApprovedVersion], [DevicesWithApp])`

• `Patch Currency n-1 = DIVIDE([DevicesOnCurrentOrPrevious], [DevicesWithApp])`

• `Stale Device Rate = DIVIDE([DevicesStale7d], [DevicesTotal])`

1. **File > Save As** → name the file `App Estate Reporting – Dataset – v1`.

Why: one dataset with one set of named measures is the single source of truth. Every visual on every page reads these measures, so no two pages can disagree.

Step 8. Build the three report views

Author three report pages in the same file. Each page answers a specific audience's question.

Page 1 — Executive view (answers "are we meeting our promises?"):

1. Three **KPI / card** visuals for the SLO: **Availability** (Install Success Rate vs 97% target), **Timeliness** (P90 time-to-deploy vs 96 h), **Currency** (Patch Currency n-1 vs 95%). Colour each by whether it meets target (conditional formatting off the KPI targets table).

1. One **error-budget gauge**: budget remaining in the rolling 30-day window.

2. One **trend line**: overall install success rate over time (needs a few days of export history to populate).

1. Keep it to a single screen. No per-app tables here — executives get status, not detail.

Page 2 — Operations view (answers "where do I spend my day?"):

1. A **bar chart**: failure count by app (top 15), sorted descending — the day's work queue.

2. A **matrix**: failure rate by **ring** × app, conditionally formatted red where a ring exceeds

3% or is more than 2× another ring.

1. A **column chart**: detection mismatch rate by app (the "installs but reports failed" apps).
2. A **card row**: stale device rate, pending installs older than 96 h.

Page 3 — Per-app drill-down (answers "what's wrong with THIS app?"):

1. A **slicer** at the top to pick one app.
2. Cards: Install Success Rate, Failure Rate, Version Drift %, Adoption (with its confidence label from the overview caveat).
1. A **table** of failing devices for the selected app: device name, ring, last check-in, install state, detection state — this is the list you hand to the troubleshooter.
1. A **version histogram**: device count by installed version, with the approved version marked.

Configure **drill-through** from Page 2's app bar to Page 3 filtered to that app, so one click on a red bar lands the operator on the drill-down for exactly that app.

Step 9. Publish, share, and schedule refresh

1. In Power BI Desktop → **Publish** → select the workspace `Endpoint & App Reporting`.

(Government cloud: sign Desktop into `app.powerbigov.us` first.)

1. The published report is named `App Estate Reporting – Report – v1` and its dataset `App Estate Reporting – Dataset – v1`.
1. In the Power BI service, open the dataset → **Settings > Scheduled refresh**:

• Configure the **data source credentials** (the Storage Blob Data Reader identity from Step 3; for an on-prem folder, install and bind an **on-premises data gateway**).

• Set **refresh twice daily** (for example 06:00 and 18:00), after the nightly export window.

1. Share to consumers via **Viewer** role on the workspace, or publish an **app** from the workspace for the wider audience.

Why: scheduled refresh from stored exports is what makes the dashboard "living" without anyone touching it, and workspace roles are how you keep the raw failing-device data on a need-to-know basis.

Step 10. Turn on the nightly export

1. Confirm the scheduled task `App Estate Reporting – Nightly Export` exists (created in Step 4).

Open **Task Scheduler** on the runner and check it points at `APPRPT-Export-AppInstallStatus.ps1` with the export identity, `-Environment` for your cloud, and the `-BlobContainer app-estate-exports` (or `-OutputPath`) target.

1. Set the trigger to run nightly at a quiet hour **before** the Power BI refresh times from Step 9.

2. Right-click → **Run** once to confirm it produces a fresh export file under the storage location

using app-only auth (no interactive prompt).

1. Confirm the next scheduled Power BI refresh picks up the new file.

Why: the whole point of the pipeline is that nobody exports by hand again. The task is the engine; Steps 5–6 already proved the payload is correct.

Step 11. Configure alerting (Azure Monitor)

Alert only on conditions a human must act on **now**; everything else waits for the weekly review (see Best Practices for the full alert-vs-review split).

1. Create the action group. **Azure portal > Monitor > Alerts > Action groups > + Create** → name

`AG – Endpoint Operations` → add the endpoint on-call as an **Email/SMS/Push** or a **webhook** to your ticketing system → **Create**.

1. Create the example alert rule against the Log Analytics workspace `law-endpoint-ops` (requires R9).

Monitor > Alerts > + Create > Alert rule → **Scope:** `law-endpoint-ops` → **Condition:** a **Custom log search** whose KQL computes install failure rate over the last 6 hours and fires when it exceeds **2×** the trailing 7-day baseline (or an absolute floor such as > 6%). Name the rule `ALERT – App Failure Rate – High`, severity **Sev 2**, action group `AG – Endpoint Operations`.

1. Add a second rule for **error-budget burn**: fire when more than half the 30-day error budget is

consumed in the first 10 days of the window.

1. **Test it:** temporarily lower the threshold so the rule fires, confirm the action group notifies,

then restore the threshold.

Why: paging on the wrong thing trains people to ignore the pager. Two well-chosen conditions — a sudden failure spike and a budget-burn — are the only two that genuinely can't wait for Monday.

Step 12. Stand up the supportability model

1. Assign each KPI an **owner** (Best Practices has the ownership table). Put the owner's name next to

the KPI in `App Estate KPI Set – v1`.

1. Create the **triage queue** — a saved view/queue in your ticketing system named **`App Estate

Triage`** — that the daily on-call works from. Alerts from Step 11 land here.

1. Define the **evidence pack** an app-failure ticket must carry (Best Practices lists the exact

artifacts): app name + ID, ring, affected device list (from Page 3), install/detection state, and the relevant `IntuneManagementExtension` log excerpt. Add these as required fields/attachments on the ticket template.

1. Put the **cadences** on calendars: daily triage (15 min), weekly operational review (against Page

2), monthly service review (against Page 1 and the SLO/error budget).

Why: a red number that has no owner and no queue is just decoration. This step is what converts the dashboard into an operating rhythm.

Step 13. Expand to the whole estate

1. Once the pilot pipeline has matched the console for a full week and the review cadences are running, widen scope: change the export scripts from the pilot app filter / `PILOT – App Estate Reporting` group to the full estate (remove the `-AppNameLike` / group filter).
1. Re-run Step 6's validation against **two or three** more apps across different rings to confirm the numbers still match at fleet scale.
1. Confirm scheduled-refresh duration and export runtime stay within their windows at full volume (Troubleshooting covers throttling and paging if they don't).
1. Announce the dashboard and the review cadences to the wider endpoint and support teams.

Why: widening only after the pilot proves out is what keeps a wrong definition from being published to the whole organization at once.

Testing & Validation

These are the pass/fail gates that prove the reporting layer is **trustworthy** — that the numbers on the dashboard match reality and that alerts and ownership actually work. Run the gates in order; do not widen scope (Implementation Step 13) until the pilot passes all of them. Each gate names the exact object from Implementation.

Gate 1 — The extractor authenticates and returns data

Procedure: Run `APPRPT-Export-AppInstallStatus.ps1 -Environment <your cloud> -OutputPath C:\AppEstateReporting\Exports -Format Both`.

Pass: A file `app-install-status_<timestamp>.csv` appears with one row per app×device and a non-zero row count that roughly matches your pilot apps × targeted devices.

Fail: Empty output or an auth error → check Graph scopes (R3), the `-Environment` value (R11), and that the pilot apps have assignments.

Gate 2 — Export counts match the Intune console (the trust gate)

Procedure: For one pilot app, read Installed / Failed / Not applicable / Pending from **Intune admin center > Apps > All apps > (app) > Monitor > Device install status**, then count the same states for that app in `app-install-status_*.csv`.

Pass: The four counts agree (allow a small drift if devices checked in between the two reads).

Fail: Material divergence → almost always a difference in how "applicable" is counted, or a stale export. See Troubleshooting T2.

Gate 3 — The KPI snapshot math is correct

Procedure: Run `APPRPT-Generate-KPISnapshot.ps1`. Take the console counts from Gate 2 and compute `installed ÷ (installed + failed)` by hand for a single app; compare to the snapshot's `InstallSuccessRatePct`.

Pass: The hand calculation equals the snapshot value, and each `*_Verdict` field is PASS/FAIL per the healthy ranges in `App Estate KPI Set – v1` (success ≥ 97%, failure ≤ 3%, stale ≤ 5%).

Fail: Mismatch → confirm you compared the same scope (whole estate vs one app) and the same snapshot time.

Gate 4 — Top-failures ranking surfaces the right apps

Procedure: Run `APPRPT-Report-TopFailures.ps1 -Top 20`. Cross-check the #1 app against the console.

Pass: The app with the most failures in the console is at or near the top of the ranked list, and any app above 3% is flagged `ATTENTION`.

Fail: A known-bad app is missing → confirm it has device-status data and is within any `-AppNameLike` filter.

Gate 5 — Version drift reflects reality

Procedure: Pick an app you know has mixed versions. Run `APPRPT-Report-VersionDrift.ps1 -AppDisplayNameLike "<app>" -ApprovedVersion "<known approved>"`. Spot-check two devices in the histogram against **Intune > Devices > (device) > Discovered apps**.

Pass: The histogram version for each spot-checked device matches the device's discovered-apps entry, and drift % / currency % carry the correct PASS/FAIL against 10% / 95%.

Fail: Versions disagree → detected-app inventory may be stale; confirm the device's last sync.

Gate 6 — The Power BI dataset refreshes from storage

Procedure: In the Power BI service, open `App Estate Reporting – Dataset – v1` → **Refresh now**. Then open `App Estate Reporting – Report – v1`.

Pass: Refresh completes without credential errors and the three pages render with current numbers that match the latest export file.

Fail: Refresh fails → check the data source credentials / gateway (Implementation Step 9) and that the export file is where the dataset expects it.

Gate 7 — The three views answer their questions

Procedure: Open each page and confirm it answers its audience's question at a glance.

Pass:

• **Executive** — the three SLO cards (Availability, Timeliness, Currency) show a value and a target colour, and the error-budget gauge is populated.

• **Operations** — the failure-by-app bar and the failure-by-ring matrix highlight the same apps the scripts flagged in Gates 4–5.

• **Drill-down** — selecting an app in the slicer filters the failing-device table and version histogram to that app, and **drill-through from the Operations bar lands on this page filtered to the clicked app**.

Fail: A page is a wall of numbers, or drill-through doesn't filter → re-check Step 8's visual and drill-through configuration.

Gate 8 — The nightly export runs unattended

Procedure: In Task Scheduler, right-click `App Estate Reporting – Nightly Export` → **Run**.

Pass: A fresh export file appears in the storage location within a few minutes, produced with **no interactive sign-in** (app-only auth), and the next scheduled Power BI refresh picks it up.

Fail: Interactive prompt or no file → the task is using the wrong identity or the app-only credential/consent is missing (R4). See Troubleshooting T4.

Gate 9 — An alert fires and reaches a human

Procedure: Temporarily lower the threshold on `ALERT – App Failure Rate – High` so it fires against current data.

Pass: The rule triggers, `AG – Endpoint Operations` notifies (email/SMS/ticket), and the item lands in the `App Estate Triage` queue. Restore the threshold afterward.

Fail: No notification → check the action group members and the alert rule's query/scope (`law-endpoint-ops`).

Gate 10 — The supportability loop closes

Procedure: Take one currently-failing app. Build the **evidence pack** from Page 3 (app + ID, ring, failing-device list, install/detection state, log excerpt) and open a ticket in `App Estate Triage` using the template.

Pass: The ticket is actionable on first read — the assigned owner can start work without asking for more information — and the KPI it maps to has a named owner in `App Estate KPI Set – v1`.

Fail: The ticket bounces for missing information → the evidence-pack fields on the template are incomplete (Implementation Step 12).

Overall exit criteria (pilot → expand)

You may proceed to Implementation Step 13 (whole estate) only when: Gates 1–8 pass for at least **one week of continuous exports**, Gate 9 has fired and notified at least once, and the daily/weekly review cadences have run at least once each against real data.

Troubleshooting

This covers the failure modes of the **reporting pipeline and dashboard themselves** — not per-install app failures, which the dashboard exists to surface and hand to a troubleshooter. Each entry is **Symptom** → **Cause** → **Fix**, and references the exact objects from Implementation.

T1. The extractor returns no rows / empty CSV

Symptom: `APPRPT-Export-AppInstallStatus.ps1` completes but writes an empty file, or prints "No app device-status data found."

Cause & fix:

- **No assignments / no installs yet** — an app with no assignment has no `deviceStatuses`. Confirm the pilot apps are assigned to devices in `PILOT – App Estate Reporting`.
- `-AppNameLike` **filter too narrow** — it silently excludes everything. Run once without the filter.
- **Wrong scopes** — if `Connect-MgGraph` succeeded but data is empty, confirm `DeviceManagementApps.Read.All` was consented (R3), not just requested.
- **Wrong cloud** — a Commercial `-Environment Global` run against a GCC High tenant authenticates to the wrong endpoint and returns nothing. Set `-Environment USGov` / `USGovDoD` (R11).

T2. Export counts don't match the Intune console (Testing Gate 2 fails)

Symptom: The console shows, say, 480 installed for an app; the CSV shows 455.

Cause & fix:

- **"Applicable" counted differently** — the console's Overview may exclude `notApplicable` devices from a percentage while your count includes them. Compare **raw state counts**, not percentages, and make sure both sides define "applicable" as `total – notApplicable`.
- **Snapshot timing** — devices check in continuously; a console read and an export minutes apart legitimately differ by a few devices. Re-read both within the same minute for the gate.

• **Beta endpoint shape changed** — the scripts use the Graph **beta** `deviceStatuses`; Microsoft occasionally changes fields. If a state value looks unmapped, inspect one raw record with `Invoke-MgGraphRequest -Method GET -Uri ".../deviceStatuses"` and adjust the state filter.

T3. Power BI refresh fails with a credentials or gateway error

Symptom: Scheduled refresh on `App Estate Reporting – Dataset – v1` fails; the service reports a credential or data-source error.

Cause & fix:

• **Azure Blob source** — set the data-source credentials on the dataset (**Settings → Data source credentials**) to an identity with** Storage Blob Data Reader** on `app-estate-exports` (R6).

• **On-prem folder source** — a folder like `C:\AppEstateReporting\Exports` needs an **on-premises data gateway** installed and bound to the dataset; a cloud-only refresh cannot reach a local path.

• **File moved/renamed** — if the dataset points at a specific file rather than a folder/container, each new dated export breaks it. Point the query at the **folder/container** and combine files, not at one filename.

T4. The nightly task prompts for sign-in or produces nothing (Testing Gate 8 fails)

Symptom: `App Estate Reporting – Nightly Export` runs but a sign-in window appears, or it produces no file when run under the service account.

Cause & fix:

• **Still using delegated auth** — the interactive extractor path prompts. The unattended run must use the app registration `App Estate Reporting – Graph Export` with **certificate + application scopes** (R2/R4). Confirm the task's script invocation uses app-only auth, not interactive `Connect-MgGraph`.

• **Consent missing** — application scopes show as *requested* but not *granted*. Grant admin consent (App registration → API permissions → **Grant admin consent**), then re-run.

• **Certificate not found / expired** — the cert must be in the store the task account can read, and in date. Re-run `APPRPT-Bootstrap-ScheduledExport.ps1` (it sets a 1-year cert) or renew it.

• **Wrong user context** — the task must run as an account that has the certificate and write access to the storage location. Check **Task Scheduler** → **the task** → **General** → **"Run as."**

T5. Version-drift numbers look wrong or the app isn't found

Symptom: `APPRPT-Report-VersionDrift.ps1` reports "App not found," or a version you know is deployed is missing from the histogram.

Cause & fix:

• **Name mismatch** — `detectedApps` uses the app's *installed* display name, which can differ from the Intune app name (e.g. "Google Chrome" vs the packaging name). Widen `-AppDisplayNameLike`.

• **Stale detected-app inventory** — Intune refreshes discovered apps on a schedule; a very recent install may not appear yet. Confirm the device's **last sync** is recent.

• **Version string parsing** — some apps report non-numeric version strings. The script sorts defensively, but treat the histogram order as approximate for such apps and rely on the counts.

T6. Graph requests throttle at fleet scale (HTTP 429)

Symptom: After Step 13 (whole estate), the extractor or KPI snapshot slows dramatically or errors with 429 / "Too many requests."

Cause & fix:

• **Expected at scale** — reading `deviceStatuses` per app across a large estate is many calls. The scripts follow `@odata.nextLink` paging; add a short delay between apps, run the nightly export in a **quiet window**, and consider the Graph `exportJobs` report API (asynchronous, one job instead of many calls) for very large tenants — see Overview data source #2.

• **Overlapping runs** — don't run the interactive extractor and the nightly task at the same time.

• **Honor Retry-After** — if you customize the scripts, respect the `Retry-After` header on a 429 rather than hammering.

T7. Alert never fires, or fires constantly (Testing Gate 9)

Symptom: `ALERT – App Failure Rate – High` either never triggers or pages every hour.

Cause & fix:

• **Never fires** — the KQL scope/query is wrong or `law-endpoint-ops` isn't receiving app data. Confirm the diagnostic setting `Intune → law-endpoint-ops` is sending the needed categories (R9) and test by lowering the threshold (Gate 9).

• **Fires constantly** — the threshold is below your normal baseline. Re-tune to **2× the trailing 7-day baseline** (or a sensible absolute floor) after observing a month of real data, per Best Practices.

• **No notification though it fired** — the action group `AG – Endpoint Operations` has no valid recipient/webhook, or the ticketing webhook rejected the payload. Re-check the action group.

T8. Government-cloud endpoint errors

Symptom: In a GCC High / DoD tenant, `Connect-MgGraph` fails, or Power BI publish targets the wrong service.

Cause & fix:

- **Graph** — pass `-Environment USGov` (Graph `graph.microsoft.us`) or `-Environment USGovDoD` (`dod-graph.microsoft.us`). The default `Global` cannot reach a sovereign tenant.
- **Power BI** — sign Power BI Desktop into `app.powerbigov.us` before publishing; a report authored against the commercial service cannot be published to the Government cloud. Keep a **separate** Government dataset/report (Best Practices → Government cloud).
- **Feature parity** — if a Graph report endpoint or Power BI feature you depend on isn't available in your Government cloud yet, **verify for GCC High** against Microsoft's US-sovereign-cloud documentation; do not assume commercial parity.

T9. Adoption numbers look implausible or can't be sourced

Symptom: The adoption KPI is blank, zero, or obviously wrong.

Cause & fix:

- **No native source** — Intune does not report per-app *launch* telemetry for Windows desktop apps. Wire adoption from whatever signal your estate has (M365 Apps usage, Defender for Endpoint software inventory, Endpoint analytics app reliability, or the vendor portal) and **label the KPI's source and confidence** (Overview caveat, Best Practices). Do not present an adoption number whose source you can't name.
- **Confidence not shown** — if the number is assembled from indirect signals, mark it "indicative" on the drill-down page so consumers read it correctly.

T10. Two visuals show different values for the "same" KPI

Symptom: The executive card and an operations visual disagree on, say, install success rate.

Cause & fix:

- **KPI recomputed from a second query** — the cardinal sin. Every visual must read the **shared DAX measure** from `App Estate Reporting – Dataset – v1`, not its own recomputation. Delete the offending ad-hoc measure and repoint the visual at the canonical measure.
- **Different filter context** — one visual is filtered (a slicer, a page filter) and the other isn't. Confirm both are at the same scope before concluding they disagree.

Best Practices

These are the disciplines that keep the reporting layer trustworthy, cheap, and actually used. They cover the two things that most often go wrong — alerting on the wrong signals, and letting the dashboard drift away from reality — plus retention, least privilege, and cost.

Alert vs. weekly review — the split that keeps the pager credible

The fastest way to make people ignore monitoring is to page them on things that could have waited. Decide, for every condition, whether a human must act **now** or whether it belongs in a scheduled review. Only two classes of condition earn a page.

Condition	Route	Why
Install failure rate spikes to **> 2× the 7-day baseline** in a 6-hour window	**Page** (`ALERT – App Failure Rate – High`)	A live rollout is going wrong; every hour multiplies the blast radius.
Error budget burn — more than half the 30-day budget consumed in the first third of the window	**Page**	The SLO will be missed unless someone intervenes; earliest honest signal.
A single app creeps from 1% to 3% failures over two weeks	**Weekly review**	Real but not urgent; belongs on the Operations page agenda.
Version drift grows past 10% on one app	**Weekly review**	Schedule an update/supersedence; not a fire.
Adoption below target for a licensed app	**Monthly service review**	A value/cost conversation, not an incident.
Stale device rate rises	**Weekly review**	Investigate check-in health; rarely time-critical.

Rule of thumb: **page on rate-of-change and budget burn; review on levels and trends.** Re-tune the `ALERT – App Failure Rate – High` threshold after the first month against your real baseline — a threshold that never fires is as useless as one that always does.

Ownership — every KPI has a name next to it

A red number with no owner is decoration. Record the owner in `App Estate KPI Set – v1`.

KPI	Owner (role)	Cadence
Install success / failure rate	Endpoint operations lead	Daily triage + weekly review
Failure rate by ring	Deployment/rollout owner	Weekly review
Detection mismatch rate	App packaging owner	Weekly review
Time-to-deploy (P50/P90)	Deployment/rollout owner	Weekly review
Version drift / patch currency	Patch/update owner	Weekly review
Uninstall success	Endpoint operations lead	Weekly review
App adoption / usage	App/service owner + finance liaison	Monthly service review
Stale device rate	Endpoint operations lead	Weekly review

KPI	Owner (role)	Cadence
SLO / error budget	Service owner	Monthly service review

The evidence pack — make every app-failure ticket actionable on first read

When a red number becomes a ticket in `App Estate Triage`, it must carry enough to act on without a follow-up round-trip. Require these fields/attachments on the template:

1. **App name + AppId** and app type (Win32 / LOB / Store).
2. **Ring / assignment** the failure clusters in.
3. **Affected device list** — exported from the drill-down page (device, last check-in, install state, detection state).
1. **Install + detection state** for a representative device.
2. **Log excerpt** — the relevant lines from `C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\IntuneManagementExtension.log` / `AppWorkload.log`, or a **Collect diagnostics** package.
1. **KPI + target** the ticket is defending (e.g. "install success 92% vs 97% SLO").

A ticket with all six is picked up and worked; a ticket with two bounces back and wastes a day.

Keep the dashboard honest

• **One source per KPI.** The single biggest cause of a distrusted dashboard is the same metric computed two ways that disagree by a few percent. The KPI table assigns exactly one source per metric — keep it that way, and never add a visual that recomputes a KPI from a different query.

• **Re-validate against the console monthly.** Repeat Testing Gate 2 for two or three apps each month. Microsoft changes report shapes and Graph beta endpoints; a quarterly drift check catches it before a leader does.

• **Label confidence on adoption.** Because Intune has no native per-app launch telemetry for Windows desktop apps, always show which source the adoption number came from and mark it "indicative" if it's assembled from Defender/M365/Endpoint-analytics signals rather than a definitive usage feed.

• **Version the KPI set.** If you change a definition or target, bump to `App Estate KPI Set – v2` and re-author the measures — never silently redefine a metric people are already tracking.

Least privilege for the reporting identity

• The unattended export identity `App Estate Reporting – Graph Export` gets **read-only application scopes only** (`DeviceManagementApps.Read.All`, `DeviceManagementManagedDevices.Read.All`). A reporting pipeline never needs write.

• Prefer a **certificate** over a client secret, and set a **1-year** expiry with a renewal reminder.

• Grant storage access at the **container** level (`app-estate-exports`), not the whole account: **Storage Blob Data Contributor** to the writer, **Storage Blob Data Reader** to the Power BI refresh identity.

• Keep failing-device detail on need-to-know: **Viewer** on the Power BI workspace for consumers, full access only for the operations team.

Data retention

• **Exports in storage:** keep 90 days of raw snapshots for reproducibility and trend backfill; expire older blobs with an Azure Storage **lifecycle management** rule to control cost.

• **KPI snapshots:** keep the appended trend CSV indefinitely — it's tiny and it's what the executive trend line and error-budget history read.

• **Log Analytics:** set the `law-endpoint-ops` retention to your compliance minimum (often 30–90 days interactive), archiving longer if required. Ingestion and retention are the main cost lever (see below).

Cost

• **Power BI Pro** per author/consumer, or **Premium/Fabric** capacity for broad read-only sharing — decide by audience size, not by feature envy.

• **Log Analytics** is billed by GB ingested and retention days. Send only the categories you alert on; app/audit categories are far cheaper than verbose device telemetry. Start narrow and add.

• **Azure Storage** for exports is negligible with a 90-day lifecycle rule.

• The whole pipeline is designed to be **read-only and low-volume** — a nightly export plus two Power BI refreshes — so it stays cheap by construction. The expensive mistake is over-collecting logs "just in case."

Government cloud

Keep a **separate** Power BI report and dataset for a Government tenant — the commercial and `app.powerbigov.us` services do not share workspaces, and feature parity lags. Author against the Government Graph and storage endpoints from the start (Requirements R11) rather than trying to re-point a commercial report later. Where a Graph report endpoint or Power BI feature you rely on is not yet in your Government cloud, **verify for GCC High** against Microsoft's US-sovereign-cloud documentation before promising the metric.

Deliverables Summary

This solution is a self-contained package for building the monitoring, reporting, and supportability layer over your Intune-managed application estate. Everything below is delivered as downloadable files you keep and reuse across unlimited devices and rollouts.

Documentation (Word + PDF)

Section	What it gives you
Solution Overview	The measurement model (delivery / velocity / currency / value + data quality), the `App Estate KPI Set – v1` with healthy ranges, the app-delivery SLO and error budget, the five data sources, the pipeline diagram, and the supportability model.
Requirements	11 requirements, each with Description / Why / Verify / Implement-if-missing — Intune roles, the export app registration, exact Graph scopes (delegated and application), Power BI Pro licensing, storage, Log Analytics + diagnostic settings, PowerShell, and the Government-cloud endpoint table.
Implementation	The exact ordered build: cloud selection, KPI/SLO sign-off, pilot scope group, storage, the least-privilege export identity, first manual export, the console trust gate, the Power BI dataset + three-view report, publish + scheduled refresh, the nightly export task, Azure Monitor alerting, the supportability model, and estate-wide expansion. Every object named.
Testing & Validation	10 pass/fail gates from "extractor authenticates" through "supportability loop closes," plus the pilot→expand exit criteria.
Troubleshooting	The common failure modes of the pipeline itself — empty exports, console/export mismatch, refresh failures, unattended-auth failures, throttling, and Government-endpoint issues.
Best Practices	The alert-vs-review split, KPI ownership table, the evidence-pack definition, dashboard-honesty rules, least privilege, retention, and cost.
Executive Summary	The one-page business case for non-technical stakeholders.

PowerShell scripts (ZIP)

Five scripts using the Microsoft Graph PowerShell SDK. The four report/extract scripts are **read-only**; the bootstrap script makes changes and is **-WhatIf capable**, previewing every action. All five accept `-Environment Global | USGov | USGovDoD` to target Commercial or US Government clouds.

Script	What it does
`APPRPT-Export-AppInstallStatus.ps1`	The backbone extractor — per-device install/detection status for every app → CSV/JSON for the Power BI dataset. Read-only.

Script	What it does
`APPRPT-Report-TopFailures.ps1`	Ranks the estate's worst deployment failures by app and rate; flags anything above the 3% healthy range. The daily triage queue. Read-only.
`APPRPT-Report-VersionDrift.ps1`	Version histogram, drift %, and patch currency (n-1) for a named app against an approved version; lists outdated devices. Read-only.
`APPRPT-Generate-KPISnapshot.ps1`	Rolls the whole estate into one `App Estate KPI Set – v1` snapshot row with PASS/FAIL verdicts; append per run to build the trend. Read-only.
`APPRPT-Bootstrap-ScheduledExport.ps1`	One-time setup: creates the read-only export app registration, grants least-privilege application scopes, and registers the nightly export task. Makes changes; `-WhatIf` capable.

What you achieve

A living, trustworthy view of application delivery health — install success, failure by app and ring, detection mismatch, time-to-deploy, version drift, adoption, and stale devices — measured the same way every time, refreshed automatically, alerting on the two conditions that can't wait, and wrapped in an ownership and triage model so every red number lands on a named owner's queue with the evidence already attached. Built once for the pilot, then widened to the whole estate.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

APPRPT-Bootstrap-ScheduledExport.ps1

Purpose: One-time setup for the automated pipeline. Creates (or reuses) the Entra app registration 'App Estate Reporting - Graph Export', adds a self-signed certificate credential, grants and admin-consents the least-privilege APPLICATION Graph scopes, then registers the Windows scheduled task 'App Estate Reporting - Nightly Export' that runs the install-status extractor with app-only auth. This script MAKES CHANGES to Entra and the local machine, so it is -WhatIf capable and previews every action first. HOW TO RUN Preview (no changes): pwsh -File .\APPRPT-Bootstrap-ScheduledExport.ps1 -Environment Global -WhatIf Apply: pwsh -File .\APPRPT-Bootstrap-ScheduledExport.ps1 -Environment Global `-ExportScriptPath C:\AppEstateReporting\APPRPT-Export-AppInstallStatus.ps1` -OutputPath C:\AppEstateReporting\Exports -RunTime 02:30 Run as GLOBAL ADMINISTRATOR (admin consent for application permissions requires it). Government cloud: -Environment USGov | USGovDoD (routes Graph + consent accordingly). Required Graph scopes to RUN this bootstrap (delegated): Application.ReadWrite.All, AppRoleAssignment.ReadWrite.All, DelegatedPermissionGrant.ReadWrite.All EXPECTED RESULTS An app registration with two read-only APPLICATION scopes admin-consented, a certificate in CurrentUser\My, and a scheduled task that produces a fresh export nightly with no interactive sign-in. Verify per Requirements R2/R4 and Implementation Step 10. .NOTES Intune Solutions - the repeatable way. Creates an unattended read-only reporting identity; every write action honors -WhatIf. >

How to run: Preview (no changes): pwsh -File .\APPRPT-Bootstrap-ScheduledExport.ps1 -Environment Global -WhatIf Apply: pwsh -File .\APPRPT-Bootstrap-ScheduledExport.ps1 -Environment Global `-ExportScriptPath C:\AppEstateReporting\APPRPT-Export-AppInstallStatus.ps1` -OutputPath C:\AppEstateReporting\Exports -RunTime 02:30 Run as GLOBAL ADMINISTRATOR (admin consent for application permissions requires it). Government cloud: -Environment USGov | USGovDoD (routes Graph + consent accordingly). Required Graph scopes to RUN this bootstrap (delegated): Application.ReadWrite.All, AppRoleAssignment.ReadWrite.All, DelegatedPermissionGrant.ReadWrite.All EXPECTED RESULTS An app registration with two read-only APPLICATION scopes admin-consented, a certificate in CurrentUser\My, and a scheduled task that produces a fresh export nightly with no interactive sign-in. Verify per Requirements R2/R4 and Implementation Step 10. .NOTES Intune Solutions - the repeatable way. Creates an unattended read-only reporting identity; every write action honors -WhatIf. >

APPRPT-Export-AppInstallStatus.ps1

Purpose: The backbone extractor. For every managed app (or a filtered subset), reads the per-device install state (installed / failed / notApplicable / pending) and, where available, the detection state, and writes a flat CSV and/or JSON snapshot that the Power BI dataset 'App Estate Reporting - Dataset - v1' consumes. Read-only; never changes Intune. HOW TO RUN Interactive (delegated): pwsh -File .\APPRPT-Export-AppInstallStatus.ps1 -Environment Global ` -OutputPath C:\AppEstateReporting\Exports -Format Both Filter to specific apps: ... -AppNameLike "Acme" Unattended (app-only) is driven by the scheduled task the bootstrap script creates. Government cloud: pass -Environment USGov (GCC High) or -Environment USGovDoD (DoD). Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS One file per run: app-install-status_<yyyyMMdd-HHmm>.csv and/or .json in -OutputPath. Each row = one app x one device with its install and detection state. Row count roughly equals (apps with assignments) x (targeted devices).

Feed this file to Power BI. .NOTES Intune Solutions - the repeatable way. Read-only against Microsoft Graph. >

How to run: Interactive (delegated): pwsh -File .\APPRPT-Export-AppInstallStatus.ps1 -Environment Global ` -OutputPath C:\AppEstateReporting\Exports -Format Both Filter to specific apps: ... -AppNameLike "Acme" Unattended (app-only) is driven by the scheduled task the bootstrap script creates. Government cloud: pass -Environment USGov (GCC High) or -Environment USGovDoD (DoD). Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS One file per run: app-install-status_<yyyyMMdd-HHmm>.csv and/or .json in -OutputPath. Each row = one app x one device with its install and detection state. Row count roughly equals (apps with assignments) x (targeted devices). Feed this file to Power BI. .NOTES Intune Solutions - the repeatable way. Read-only against Microsoft Graph. >

APPRPT-Generate-KPISnapshot.ps1

Purpose: Rolls the whole app estate up into one KPI snapshot: install success rate, failure rate, detection mismatch rate, stale device rate, and applicable/installed/failed totals, each with its healthy-range PASS/FAIL verdict. Appending one snapshot per run builds the trend that the executive dashboard and the SLO error budget read. Read-only. HOW TO RUN pwsh -File .\APPRPT-Generate-KPISnapshot.ps1 -Environment Global -OutputPath <path> Optional: -AppNameLike "Acme" to scope; -Append <file.csv> to grow a trend file. Government cloud: -Environment USGov | USGovDoD. Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS One snapshot row (timestamp + each KPI + PASS/FAIL) printed and written to CSV/JSON. Run it on the same schedule as the export so the trend line stays continuous. Pass = all KPIs in their healthy range; any FAIL is the weekly review agenda. .NOTES Intune Solutions - the repeatable way. Read-only against Microsoft Graph. >

How to run: pwsh -File .\APPRPT-Generate-KPISnapshot.ps1 -Environment Global -OutputPath <path> Optional: -AppNameLike "Acme" to scope; -Append <file.csv> to grow a trend file. Government cloud: -Environment USGov | USGovDoD. Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS One snapshot row (timestamp + each KPI + PASS/FAIL) printed and written to CSV/JSON. Run it on the same schedule as the export so the trend line stays continuous. Pass = all KPIs in their healthy range; any FAIL is the weekly review agenda. .NOTES Intune Solutions - the repeatable way. Read-only against Microsoft Graph. >

APPRPT-Report-TopFailures.ps1

Purpose: Produces the operations work queue: the top-N apps by failure count and failure rate, a by-ring breakdown (which deployment ring is worst), and the affected device list for the worst apps. This is the number that drives the daily triage. Read-only. HOW TO RUN pwsh -File .\APPRPT-Report-TopFailures.ps1 -Environment Global -Top 20 -OutputPath <path> Optional: -AppNameLike "Acme" to scope; -RingGroupPrefix "RING -" to derive rings from group names; -CsvPath out.csv to save. Government cloud: -Environment USGov | USGovDoD. Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A ranked table (AppName, Failed, Applicable, FailureRatePct) sorted by failures, plus the by-ring failure breakdown. Pass = low/steady failures. Fail = apps or rings above the 3% healthy range in the KPI set - take those into per-app drill-down. .NOTES Intune Solutions - the repeatable way. Read-only against Microsoft Graph. >

How to run: pwsh -File .\APPRPT-Report-TopFailures.ps1 -Environment Global -Top 20 -OutputPath <path> Optional: -AppNameLike "Acme" to scope; -RingGroupPrefix "RING -" to derive rings from group names; -CsvPath out.csv to save. Government cloud: -Environment USGov | USGovDoD. Required Graph scopes: DeviceManagementApps.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A ranked table (AppName, Failed, Applicable, FailureRatePct) sorted by failures, plus the by-ring failure breakdown. Pass = low/steady failures. Fail = apps or rings above the 3% healthy range in the KPI set - take those into per-app drill-down. .NOTES Intune Solutions - the repeatable way. Read-only against Microsoft Graph. >

APPRPT-Report-VersionDrift.ps1

Purpose: Answers "how far behind the approved version is the fleet?" For a named application, reads the detected-app inventory across managed devices, groups devices by installed version, computes version drift % and patch currency (n-1) against an approved version you supply, and lists the outdated devices. Read-only. HOW TO RUN pwsh -File .\APPRPT-Report-VersionDrift.ps1 -Environment Global ` -AppDisplayNameLike "Google Chrome" -ApprovedVersion "126.0.6478.127" -OutputPath <path> Omit -ApprovedVersion to have the script treat the highest observed version as approved. Government cloud: -Environment USGov | USGovDoD. Required Graph scopes: DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A version histogram (Version, DeviceCount), the drift % and patch-currency % against the approved version, and a list of outdated devices. Pass = drift <= 10% and currency >= 95% (the KPI healthy range). Fail = below that - schedule the update / supersedence. .NOTES Intune Solutions - the repeatable way. Read-only against Microsoft Graph. >

How to run: pwsh -File .\APPRPT-Report-VersionDrift.ps1 -Environment Global ` -AppDisplayNameLike "Google Chrome" -ApprovedVersion "126.0.6478.127" -OutputPath <path> Omit -ApprovedVersion to have the script treat the highest observed version as approved. Government cloud: -Environment USGov | USGovDoD. Required Graph scopes: DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A version histogram (Version, DeviceCount), the drift % and patch-currency % against the approved version, and a list of outdated devices. Pass = drift <= 10% and currency >= 95% (the KPI healthy range). Fail = below that - schedule the update / supersedence. .NOTES Intune Solutions - the repeatable way. Read-only against Microsoft Graph. >